



DonutDuck

Complete setup and usage guide

What's in here

This guide takes you from an empty server to a fully configured bot. Read it in order the first time; after that, jump to whichever section you need.

What DonutDuck is	An overview before the instructions start
1. Installing and running	Python, dependencies, token, first launch
2. First-time setup	The five commands that make everything else work
3. Tickets	Types, panels, transcripts, the index
4. Applications	Forms, panels, reviewing
5. Giveaways	Entry rules, claiming, Split or Steal, Double or Keep
6. Staff tools	Activity checks, strikes, legit votes
7. Moderation	Warns, timeouts, bans, cases
8. Vouches	The vouch button and channel
9. STScript	Writing custom commands
10. The rest	Twitch, stats, server features, small tools
11. The dashboard	Web interface and FreeDNS
12. Troubleshooting	Things that go wrong, and why
13. Full command reference	All 157 commands

What DonutDuck is

DonutDuck is a Discord bot built for a DonutSMP community server. It does the jobs a busy Minecraft server's Discord needs doing — handling support requests, running giveaways, processing staff applications, keeping moderators accountable and moderating members — and it looks up live DonutSMP player stats on top of that.

It is one bot rather than six because these jobs overlap. A giveaway winner needs a private channel to claim in, which is a ticket. Whoever handles that claim deserves a vouch. The staff who read the tickets are the same staff the activity checks cover. Running these as separate bots means the same staff list configured five times, five different ideas of who counts as staff, and no way for one to hand off to another.

What it does

Command	What it does
Tickets	Private support channels opened from a dropdown, with per-type questions, transcripts and a searchable index
Applications	Structured forms up to 25 questions long, with an accept or deny queue and automatic role assignment
Giveaways	Timed draws with entry rules, claim tickets, and two optional minigames for the prize
Staff tools	Activity checks that strike anyone who misses them, plus legit votes recorded properly rather than counted off reactions
Moderation	Warn, timeout, kick, ban, purge and channel locks, all logged as numbered cases
Vouches	A reputation record, posted automatically at the end of a giveaway payout
STScript	A small scripting language so staff can add their own commands without touching the code
DonutSMP stats	Balances, leaderboards, auction prices and server status, scraped from donutstats.org
Server features	Welcome messages, autorole, self-assign role menus, starboard, polls and reminders
A web dashboard	An optional browser view of everything above

How it is built

Python, using discord.py. Everything persists in a single SQLite file next to the bot, so there is no database server to run and a backup is one file. Features are organised as cogs — one file per area — and the shared pieces (the staff definition, the stats scraper, the scripting language) live in their own modules so they behave the same everywhere they are used.

Three decisions shape most of what you will notice while using it:

- **One definition of staff.** `/staffroles` decides ticket visibility, giveaway access, activity checks and strikes together. Configure it once.
- **Things survive restarts.** Giveaway timers, reminders, trackers, panel dropdowns and pending claims are all stored, not held in memory. Stopping the bot mid-giveaway does not lose it.

- **Failures explain themselves.** Where Discord would return a bare permission error, the bot says which permission is missing and which command fixes it.

What it does not do

It cannot run in-game DonutSMP commands. There is no endpoint that executes a command on the Minecraft server, so /pay, /sethome and similar stay in game. Player statistics are read from a community website that rounds its numbers and refreshes roughly every half hour, so figures are indicative rather than exact.

Who runs what

Command	What it does
Members	Open tickets, apply, enter giveaways, vouch, run scripts, check stats
Staff	Everything above, plus running and handling tickets, giveaways and applications
Manage Server	Configuration: staff roles, panels, channels, forms, ticket types
Discord permissions	Moderation follows Ban Members, Kick Members and similar, not the bot's staff list

1. Installing and running

DonutDuck runs on your own machine or a small server. It needs Python 3.10 or newer and about 250 MB of disk space once dependencies are installed.

Get the files ready

- 1 Unzip **donutduck.zip**. Open a terminal in the folder that contains `donutduck/`, `README.md` and `requirements.txt` — not inside `donutduck/` itself.
- 2 Check your Python version with `python3 --version`. It must be 3.10 or higher.
- 3 Install the dependencies.

```
pip install -r requirements.txt

# Linux may need:
pip install -r requirements.txt --break-system-packages
```

Create the Discord application

- 1 Go to **discord.com/developers/applications** and click New Application. Name it DonutDuck.
- 2 Open the **Bot** tab, click Reset Token, and copy the token. You only see it once.
- 3 Still on the Bot tab, decide about **Server Members** under Privileged Gateway Intents. Turning it on lets activity checks find staff automatically. See the callout below.
- 4 Open **OAuth2 > URL Generator**. Tick the scopes `bot` and `applications.commands`.
- 5 Under bot permissions tick: View Channels, Send Messages, Embed Links, Attach Files, Read Message History, Manage Channels, Manage Roles, Manage Messages, Kick Members, Ban Members, Timeout Members.
- 6 Open the generated URL and add the bot to your server.

About the Server Members intent

Leave it off and everything works except automatic staff detection — you would add staff to activity checks with `/activityroster add` instead. If you do turn it on in the portal, you must also set `MEMBERS_INTENT=true` in `.env`. Setting that without enabling the portal toggle stops the bot connecting at all.

Configure and launch

- 1 Copy `.env.example` to `.env` and open it.
- 2 Paste your token after `DISCORD_TOKEN=`.
- 3 Set `GUILD_ID=` to your server's ID. Enable Developer Mode in Discord settings, then right-click your server and Copy Server ID. Without this, new commands are global and can take an hour to appear.
- 4 Run the bot.

```
python3 -m donutduck.bot
```

You should see each cog loading, the database being created, and *Logged in as DonutDuck*. Try `/help` in Discord. Ctrl+C stops it.

Keeping it running

The bot only runs while that terminal is open. Closing it stops the 12-hour trackers, giveaway timers and reminder checks. For 24/7 use a small VPS with systemd, screen or tmux. Keep `donutduck.db` on persistent storage: it holds every ticket, giveaway, application and setting.

2. First-time setup

Five commands turn a freshly invited bot into a working one. Run them in this order.

Step 1 — say who your staff are

```
/staffroles add role:@staff
```

This is the most important command in the bot. One list decides who can see tickets, run giveaways, answer activity checks and receive strikes. Add every role that should count as staff. Members with Manage Server always count, so admins can't lock themselves out.

Step 2 — set up tickets

```
/ticketsetup category:#Tickets log_channel:#ticket-logs  
/tickettype role key:support role:@staff  
/ticketpanel post channel:#support
```

The first command seeds four ticket types: Staff Application, Partner Manager Application, Support, and Giveaway Claim. The second says who handles each type. The third posts the dropdown members use.

Step 3 — set up applications

```
/application setup review_channel:#app-reviews accept_role:@staff  
/application panel post channel:#applications
```

Step 4 — choose your channels

```
/vouchchannel channel:#vouches  
/legitchannel channel:#are-we-legit  
/modlog channel:#mod-logs warn_limit:3
```

Step 5 — check it worked

```
/docs  
/ticketindex  
/staffroles list
```

If you add a staff role later

Run `/ticketsync` afterwards. Discord does not retroactively apply channel permissions, so tickets opened before the role existed will not show it without that command.

3. Tickets

Tickets are private channels opened from a dropdown panel. Each type can ask its own questions, notify its own staff role, and post its own opening message.

Ticket types

Four exist after `/ticketsetup`. Add your own with no code required — a type is just a database row:

```
/tickettype add key:media label:"Media Application" emoji:CAMERA staff_role:@staff \
questions:"Channel link?|Subscriber count?|Why us?"
```

Questions are separated by a vertical bar and become a form shown before the channel opens. Discord caps a form at five questions; the command tells you if you exceed it. Built-in types can be disabled but not deleted, so a server can't accidentally remove the giveaway-claim plumbing.

Panels

A panel is a dropdown. You can have several, each offering a different set of types:

```
/ticketpanel create key:support title:"Need help?" types:support \
channel:#support
/ticketpanel create key:apps title:"Join the team" types:staff,pm \
channel:#applications
/ticketpanel list
```

After adding or disabling a type

Run `/ticketpanel refresh`. Discord stores dropdown options on the message itself, so existing panels will not show a new type until they are redrawn.

Customising what a ticket says

```
/tickettype message key:support \
title:"Hi {name}! Ticket #{ticket}" \
message:"Thanks for contacting {server}.\nSomeone from {staff} will be with you shortly." \
colour:Yellow
```

Placeholders: `{user}` `{name}` `{tag}` `{ticket}` `{type}` `{reason}` `{server}` `{staff}`. Type `\n` for a line break, because slash commands cannot contain real newlines. Preview it with `/tickettype preview` before anyone sees it.

Reasons, the index and transcripts

- Every ticket gets a one-line **reason**, taken from whichever answer asks why. On an application form the first question is usually age, so the bot looks for wording like 'why', 'reason' or 'need help' instead of blindly using answer one.
- An **index message** is posted and pinned at the top of each ticket, and appears in the transcript too.
- `/ticketindex` lists every ticket with its reason, filterable by open, closed or type.
- Closing posts a **transcript** to your log channel and DMs the opener a copy, then deletes the channel.

Running a ticket

Command	What it does
<code>/close [reason]</code>	Close it, with transcript
<code>/ticketrename <name> [ticket]</code>	Rename from anywhere, by number or channel
<code>/ticketadd /ticketremove</code>	Add or remove someone from a ticket
<code>/tickets</code>	All open tickets
<code>/ticketindex [show] [type]</code>	Every ticket with its reason

Inside a ticket there is also a **More actions** dropdown for posting a transcript without closing, re-showing the index, and flagging the ticket as high priority (which renames the channel so it stands out in the sidebar).

4. Applications

Applications are structured forms with an accept or deny decision attached — different from a ticket, which is a conversation.

Creating forms

```
/application setup review_channel:#app-reviews accept_role:@staff

/application create key:pm label:"Partner Manager" \
review_channel:#app-reviews \
questions:"Your age and timezone?|Experience with partnerships?|Hours per week?|Why you?" \
accept_role:@partner-manager cooldown_days:14
```

Forms can hold 25 questions

Discord modals only allow five inputs, so longer forms are split into pages of five. Submitting a page saves it and offers a Continue button that opens the next one. Part-finished applications are held for 30 minutes and then dropped.

Panels

```
/application panel create key:staff-apps title:"Join the team" \
forms:staff,pm channel:#applications
/application panel refresh
```

Members can also use `/apply` directly. Both routes run the same checks, so the one-pending-application rule and the reapply cooldown apply either way.

Reviewing

Submissions arrive in your review channel with Accept and Deny buttons, restricted to Manage Server. Accepting assigns the role automatically. Both outcomes DM the applicant, and denials can carry a reason. The decision is written back onto the review post, so there is a record in context.

- One pending application per person per form.
- Denied applicants wait out a cooldown (14 days by default) before reapplying.
- Two staff clicking at once cannot both resolve the same application.
- `/applications` shows the pending queue.

5. Giveaways

```
/giveaway start prize:"$5M Coins" duration:2h winners:1 value:5000000
```

Durations are written like 30m, 2h, 3d, 1d12h, 1w. Entries live in the database, so counts survive restarts and the entry button toggles — pressing it again leaves.

Entry rules

Command	What it does
required_role	Only that role may enter
min_account_days	Blocks fresh alt accounts, checked at entry time
bonus_role + bonus_entries	Gives a role extra tickets in the draw
claim_hours	How long a winner has to claim (0 = unlimited)

Winners are only drawn from entrants still in the server who still hold any required role, so a prize is never awarded to someone who left.

Claiming

When a giveaway ends, winners press **Claim prize**. That asks for their Minecraft IGN, then opens a private claim ticket named after the winner, with the host and prize in the channel topic. Tickets are not opened automatically — that used to leave empty channels behind for winners who never returned.

Split or Steal

```
/giveaway start prize:"$10M" duration:1h winners:2 mode:"Split or Steal"
```

Requires exactly two winners. Each picks privately, hidden until both have chosen. Both split, they share it. One steals, that player takes everything. Both steal, nobody gets anything. Someone who never chooses counts as having split, because defaulting to steal would reward ignoring the prompt.

Double or Keep

```
/giveaway start prize:"$5M" duration:1h mode:"Double or Nothing" \
value:5000000
```

```
/giveawaycap (set a ceiling first, via /giveaway claimdefault menu)
```

The winner keeps the prize, which opens their claim ticket, or doubles it — which starts a brand-new giveaway worth twice as much that they must win again like everyone else.

Doubling needs a number

A prize is free text, so there is nothing to double unless you say what it is worth. Set `value:5000000`, or name the prize something like \$5m and it is read from that. With no value the Double button is not shown at all. A server cap and a five-double ceiling stop it compounding: five rounds turns \$1M into \$32M.

Managing and reviewing

Command	What it does
<code>/giveaway getdata</code>	Tabbed browser: overview, running, ended, winners, unclaimed
<code>/giveaway edit <id></code>	Change prize or winners, or extend the time
<code>/giveaway cancel <id></code>	End with no winners
<code>/giveaway reroll <id></code>	Redraw, never repeating a previous winner
<code>/giveaway claimtime <id></code>	Change the claim window on a live giveaway

6. Staff tools

Activity checks

```
/activitycheck hours:24 channel:#staff strike:true
```

Every staff role is pinged and each person presses one button. When the deadline passes, anyone who did not respond gets a strike automatically, and the result is posted with who missed it and their new totals.

If it says it cannot see your staff

The bot can only read who holds a role when the Server Members intent is enabled. Without it, use `/activityroster add` to list staff manually — the roster also fills itself in, since anyone who answers a check is remembered. `/activityroster list` shows exactly who a check would cover.

Strikes

Command	What it does
<code>/strikes [member]</code>	Someone's strike record
<code>/strike <member> <reason></code>	Give one manually
<code>/strikeclear <member> [id]</code>	Clear one or all
<code>/strikelist</code>	Everyone with strikes
<code>/strikelimit <n></code>	How many before they are flagged

Legit votes

```
/legit giveaway_id:12
```

Posts Legit and Not-legit buttons to your voting channel, with the tally recorded in the database rather than counted by hand from reactions. One vote each, changeable. Passing a giveaway ID fills in the host and prize automatically. `/legitresults` shows the score.

7. Moderation

These use Discord's own permissions — Ban Members, Kick Members, Timeout Members, Manage Messages — rather than the bot's staff list, so moderation follows what your server already grants.

Command	What it does
<code>/warn <member> <reason></code>	Warns and DMs them, counts toward the limit
<code>/timeout <member> <duration></code>	Discord timeout, max 28 days
<code>/untimeout <member></code>	Lift it early
<code>/kick <member> [reason]</code>	Remove them
<code>/ban <member> [delete_days]</code>	Ban, optionally clearing recent messages
<code>/unban <user_id></code>	Reverse a ban
<code>/purge <amount> [member]</code>	Bulk delete recent messages
<code>/slowmode <seconds></code>	Channel slowmode
<code>/lock /unlock</code>	Stop or allow posting
<code>/modlogs <member></code>	Their full history
<code>/case <number></code>	One case in detail
<code>/clearwarn <number></code>	Clear a case from the record

Two protections worth knowing

Role hierarchy is checked in both directions: you cannot action anyone at or above your own top role, and nothing works on someone above the bot's. Skipping either check produces a confusing half-failure where the command looks fine and Discord silently refuses. Separately, `/unlock` resets the permission to neutral rather than granting Send Messages, so it cannot accidentally open a channel to people a category deliberately denied.

Actions become numbered cases per server, so staff can quote case *#14*. Targets are DMed before a kick or ban, since afterwards there is no shared channel left; if their DMs are closed the response says so rather than failing quietly.

8. Vouches

```
/vouchchannel channel:#vouches  
/vouch user:@someone message:paid instantly, smooth trade
```

Posts to the vouch channel in the form:

```
@voucher: vouch @target their message
```

Giveaway claim tickets also get a **Vouch** button, which appears when the ticket is closed — by then the payout has actually happened. Those tickets wait 60 seconds before deleting so there is time to press it.

- The button uses a real user picker, not a typed name: a vouch aimed at the wrong person is worse than none.
- Mentions render but do not ping, since a busy vouch channel would otherwise notify two people per entry.
- Self-vouching, vouching bots, and repeat vouches for the same person within 24 hours are all rejected.
- `/vouches` and `/vouchtop` read the stored records.

9. STScript — custom commands

STScript is a small Scratch-like language for writing your own commands. Staff write them; anyone can run them with `/st`.

```
/stscript create name:welcome
```

That opens an editor. A script looks like this:

```
say Hello {user.name}!
set coins to 100
add 50 to coins
random roll from 1 to 6
if {roll} >= 4
say You rolled {roll} and have {coins} coins - lucky!
else
say You rolled {roll}. Unlucky.
end
repeat 3
say Reminder {loop}
end
```

Run it with `/st welcome` or `/st 1`.

The language

Command	What it does
say / reply / dm	Send a message, privately, or by DM
embed <title> <body>	Send an embed
set <name> to <value>	Numbers, text, or maths like $5 * 3$
add <n> to <name>	Increase a number
random <name> from 1 to 10	Pick a random number
repeat <n> ... end	Loop, with {loop} as the counter
while <a> <op> ... end	Loop while true
if <a> <op> ... else ... end	Branch
wait <seconds>	Pause, 10 seconds total per script
role add / remove <role>	Needs Manage Roles
stop	End early

Built-in variables: `{user}` `{user.name}` `{user.id}` `{server}` `{channel}` `{args}` `{arg1}` and so on. Operators: `==` `!=` `>` `<` `>=` `<=` `contains` `is` and `is not`.

Why it is safe to let staff write these

There is no eval, exec, import or attribute access anywhere in the interpreter. Scripts are parsed into a fixed set of node types and walked by code that only understands those types, so text like `__import__('os')` is stored as an inert string. Everything is bounded too: 300 lines, 2000 steps, 500 loop iterations, 15 messages and 5 seconds per run. A runaway loop fails with a readable error instead of hanging the bot.

10. The rest

DonutSMP stats

Player stats come from donutstats.org, a community site — no API key needed. `/stats`, `/bal`, `/leaderboard`, `/ah`, `/price` and `/status` all read from it. Values are shown rounded on that site, so figures are less precise than exact numbers, and it refreshes roughly every 30 minutes.

`/track money <player>` watches a stat and reports the change every 12 hours.

Twitch notifications

```
/twitch add streamer:somebody channel:#live role:@notify
```

Needs free credentials from dev.twitch.tv in `TWITCH_CLIENT_ID` and `TWITCH_CLIENT_SECRET`. Announcements are keyed on the stream ID, so a mid-stream disconnect does not fire a second ping.

Server features

Command	What it does
<code>/welcome set / goodbye / autorole</code>	Greetings and a role for joiners
<code>/rolemenu <key> <roles></code>	Self-assign roles from a dropdown
<code>/starboard [channel] [stars]</code>	Repost popular messages
<code>/poll <question> <options></code>	Native Discord poll
<code>/remind <when> <text></code>	Reminders that survive restarts
<code>/serverprofile avatar</code>	A different bot avatar in this server only

Small tools

`/timestamp` builds codes that show in each person's own timezone — useful for announcing giveaway times. Also `/avatar`, `/roleinfo`, `/membercount`, `/firstmessage`, `/snipe`, `/afk`, `/stealemoji`, `/roll` and `/choose`.

11. The dashboard

An optional web interface showing tickets, applications, giveaways and streams across every server you manage.

```
DISCORD_CLIENT_ID= # OAuth2 tab of the same application
DISCORD_CLIENT_SECRET=
DASHBOARD_BASE_URL= # e.g. https://yourname.mooo.com
DASHBOARD_PORT=8080
DASHBOARD_SECRET= # python -c "import secrets;print(secrets.token_hex(32))"
```

Add your base URL plus `/callback` as an OAuth2 redirect in the developer portal, or login will fail.

Login is Discord OAuth rather than a shared password, because the dashboard shows ticket reasons and application answers and so has to know who you are. You only see servers where you hold Manage Server and the bot is present.

FreeDNS

FreeDNS gives you a hostname, not hosting

You still run the bot yourself. What FreeDNS solves is a changing IP address. Take the Direct URL for your subdomain from freedns.afraid.org/dynamic/ and put it in `FREEDNS_UPDATE_URL`; the bot refreshes the record every 10 minutes. You still need to forward the port and put a reverse proxy such as Caddy in front for HTTPS.

12. Troubleshooting

Command	What it does
Commands do not appear in Discord	Set <code>GUILD_ID</code> in <code>.env</code> to your server ID and restart. Without it commands are global and can take up to an hour. Global commands also stay cached for a while.
Staff cannot see tickets	Run <code>/staffroles add role:@staff</code> , then <code>/ticketsync</code> . Discord does not retroactively apply permissions to channels that already exist.
Staff cannot run giveaways	Same fix: <code>/staffroles add</code> . Giveaway commands read that list.
Activity check says it cannot see staff	The Server Members intent is off. Either enable it in the portal and set <code>MEMBERS_INTENT=true</code> , or use <code>/activityroster add</code> .
Bot will not start, mentions privileged intents	<code>MEMBERS_INTENT=true</code> is set but the portal toggle is off. Enable Server Members in the developer portal, or set the variable back to false.
A ticket panel dropdown does nothing	The panel message is older than the bot's current configuration. Run <code>/ticketpanel refresh</code> .
New ticket type missing from the panel	Same: <code>/ticketpanel refresh</code> . Discord stores dropdown options on the message.
Cannot rename a ticket a third time	Discord rate limits channel renames to twice per 10 minutes per channel. Wait and try again.
The Double button is missing	That prize has no value set. Use <code>value:</code> when starting, or name the prize like <code>\$5m</code> .
Player stats look wrong or stale	<code>donutstats.org</code> rounds its numbers and refreshes about every 30 minutes. If a field is blank the site may have renamed a label; everything that reads it lives in <code>scraper.py</code> .
Bot stops when I close the terminal	Expected. Use <code>systemd</code> , <code>screen</code> or <code>tmux</code> , or host it on a VPS.

Where your data lives

Everything is in `donutduck.db` next to the bot: tickets, giveaways, applications, scripts, vouchers, strikes and all settings. Back that file up. Deleting it resets the bot completely.

13. Full command reference

All 157 commands, grouped by area. Arguments in <angle brackets> are required; those in [square brackets] are optional. In Discord, `/docs search:<word>` finds any of these instantly.

Tickets

Command	What it does
<code>/close [reason]</code>	Close the ticket in this channel
<code>/ticketadd <member></code>	Add a member to this ticket
<code>/ticketindex [show] [type_key]</code>	Index of tickets and the reason for each
<code>/ticketpanel create <key> <title> [types] [description] [placeholder] [channel]</code>	Save a new panel
<code>/ticketpanel delete <panel> [remove_message]</code>	Delete a saved panel
<code>/ticketpanel edit <panel> [title] [types] [description] [placeholder]</code>	Change a saved panel
<code>/ticketpanel list</code>	List saved ticket panels
<code>/ticketpanel post [panel] [channel]</code>	Post a panel into a channel
<code>/ticketpanel refresh [panel]</code>	Redraw a posted panel after changing types
<code>/ticketremove <member></code>	Remove a member from this ticket
<code>/ticketrename <name> [ticket]</code>	Rename a ticket from anywhere
<code>/tickets</code>	List open tickets in this server
<code>/ticketsetup <category> <log_channel> [max_open]</code>	Set up tickets and seed the default types
<code>/ticketsync</code>	Add staff roles to existing open tickets
<code>/tickettype add <key> <label> [description] [emoji] [staff_role] [questions]</code>	Add a new ticket type
<code>/tickettype list</code>	List this server's ticket types
<code>/tickettype message <key> [title] [message] [colour] [image] [footer]</code>	Customise what's posted when this ticket opens
<code>/tickettype pings <key> [ping_opener] [ping_staff] [show_answers]</code>	Choose who gets pinged when a ticket opens
<code>/tickettype preview <key></code>	See the opening message for a ticket type
<code>/tickettype questions <key> [questions]</code>	Set a type's application questions
<code>/tickettype remove <key></code>	Delete a custom ticket type

Command	What it does
<code>/tickettype resetmessage <key></code>	Restore the default opening message
<code>/tickettype role <key> <role></code>	Set the staff role for a type
<code>/tickettype toggle <key> <enabled></code>	Enable or disable a ticket type

Giveaways

Command	What it does
<code>/giveaway cancel <giveaway_id></code>	Cancel a giveaway without drawing
<code>/giveaway claimdefault <hours></code>	Set this server's default claim window
<code>/giveaway claimtime <giveaway_id> <hours></code>	Change the claim window
<code>/giveaway edit <giveaway_id> [prize] [winners] [extend] [description]</code>	Change a running giveaway
<code>/giveaway end <giveaway_id></code>	End a running giveaway early
<code>/giveaway entries <giveaway_id></code>	How many people entered a giveaway
<code>/giveaway getdata</code>	Browse all giveaway data in tabs
<code>/giveaway list</code>	Show running giveaways
<code>/giveaway reroll <giveaway_id> [count]</code>	Draw replacement winners
<code>/giveaway start <prize> <duration> [winners] [description] [required_role] [channel] [mode] [claim_hours] [min_account_days] [bonus_role] [bonus_entries] [value]</code>	Start a giveaway

Applications

Command	What it does
<code>/application create <key> <label> <review_channel> <questions> [accept_role] [cooldown_days]</code>	Create another application form
<code>/application delete <key></code>	Delete an application form
<code>/application list</code>	Show this server's application forms
<code>/application panel create <key> <title> [forms] [description] [placeholder] [channel]</code>	Save an application panel
<code>/application panel delete <panel></code>	Delete a saved panel

Command	What it does
/application panel edit <panel> [title] [forms] [description] [placeholder]	Change a saved panel
/application panel list	Saved application panels
/application panel post [panel] [channel]	Post a panel into a channel
/application panel refresh [panel]	Redraw posted panels
/application questions <key> <questions>	Replace a form's questions
/application setup <review_channel> [accept_role]	Create a staff application with default questions
/application toggle <key> <open>	Open or close applications
/applications	List pending applications
/apply <form>	Apply for a position

Staff

Command	What it does
/activitycheck [hours] [channel] [strike]	Ping staff to confirm they're active
/activityroster add <member>	Add someone to the roster
/activityroster list	Show who activity checks cover
/activityroster remove <member>	Remove someone from the roster
/activityroster sync	Fill the roster from your staff roles
/legit [member] [prize] [giveaway_id] [channel]	Post a legit check to the voting channel
/legitchannel <channel>	Set the legit voting channel
/legitresults <vote_id>	Results of a legit vote
/staffroles add <role>	Add a staff role
/staffroles list	Show the staff roles
/staffroles remove <role>	Remove a staff role
/strike <member> <reason>	Give someone a strike
/strikeclear <member> [strike_id]	Clear strikes
/strikelimit <limit>	Strikes before it's flagged
/strikelist	Everyone with strikes
/strikes [member]	See someone's strikes

Moderation

Command	What it does
<code>/ban <member> [reason] [delete_days]</code>	Ban a member
<code>/case <number></code>	Look up one case
<code>/clearwarn <number></code>	Clear a case from the record
<code>/kick <member> [reason]</code>	Kick a member
<code>/lock [reason]</code>	Stop members posting here
<code>/modlog <channel> [warn_limit]</code>	Set the moderation log channel
<code>/modlogs <member></code>	A member's moderation history
<code>/purge <amount> [member]</code>	Bulk delete recent messages
<code>/slowmode <seconds></code>	Set channel slowmode
<code>/timeout <member> <duration> [reason]</code>	Time a member out
<code>/unban <user_id> [reason]</code>	Unban someone by user ID
<code>/unlock</code>	Allow posting here again
<code>/untimeout <member> [reason]</code>	Remove a timeout
<code>/warn <member> <reason></code>	Warn a member

Vouches

Command	What it does
<code>/vouch <user> [message]</code>	Vouch for someone
<code>/vouchchannel <channel></code>	Set where vouches are posted
<code>/vouchdelete <vouch_id></code>	Remove a vouch (staff)
<code>/vouches [user]</code>	See someone's vouches
<code>/vouchtop</code>	Most vouched members

STScript

Command	What it does
<code>/st <script> [args]</code>	Run a custom STScript command
<code>/stscript create <name></code>	Write a new script
<code>/stscript delete <script></code>	Delete a script

Command	What it does
<code>/stscript edit <script></code>	Change a script's code
<code>/stscript help</code>	How to write STScript
<code>/stscript list</code>	All scripts in this server
<code>/stscript settings <script> [enabled] [staff_only_flag] [description]</code>	Enable, disable or restrict a script
<code>/stscript show <script></code>	View a script's code
<code>/stscript test <code></code>	Run a script without saving it

DonutSMP stats

Command	What it does
<code>/skin <player></code>	Render a player's Minecraft skin
<code>/stats <player></code>	Show a player's DonutSMP stats

Auction house

Command	What it does
<code>/ah [item]</code>	Browse DonutSMP auction house items
<code>/price <item></code>	Check an item's cheapest price

Leaderboard

Command	What it does
<code>/leaderboard <board> [page]</code>	View a DonutSMP leaderboard
<code>/rank <player> <board> [depth]</code>	Find where a player sits on a leaderboard

Tracking

Command	What it does
<code>/history <stat> <player></code>	Show recorded snapshots for a tracked stat
<code>/track <stat> <player> [channel]</code>	Track a player's stat and report changes every 12 hours
<code>/trackers</code>	List this server's active trackers
<code>/untrack <stat> <player></code>	Stop tracking a stat for a player

In-game command mirrors

Command	What it does
/bal <player>	Money for a player (mirrors /bal)
/baltop [page]	Richest players (mirrors /baltop)
/blocksbroken <player>	Blocks broken for a player (mirrors /stats)
/blocksplaced <player>	Blocks placed for a player (mirrors /stats)
/deaths <player>	Deaths for a player (mirrors /deaths)
/donut [command]	Look up what any DonutSMP command does
/kills <player>	Kills for a player (mirrors /kills)
/mobskilled <player>	Mobs killed for a player (mirrors /stats)
/playtime <player>	Playtime for a player (mirrors /playtime)
/shards <player>	Shards for a player (mirrors /shards)

Server features

Command	What it does
/rolemenu <key> <roles> [title] [description] [max_choices] [channel]	Post a self-assign role menu (dropdown)
/rolemenudelete <key>	Remove a role menu
/starboard [channel] [stars] [emoji]	Configure the starboard
/welcome autorole [role]	Role given to every new member
/welcome goodbye <channel> [message]	Set the goodbye message
/welcome off	Turn off welcome and goodbye messages
/welcome set <channel> [message]	Set the welcome message

Utilities

Command	What it does
/embed <title> <description> [colour] [channel] [image]	Post a custom embed as the bot
/poll <question> <options> [hours] [multiple]	Create a poll
/remind <when> <text>	Set a reminder
/reminders [cancel]	List or cancel your reminders
/serverinfo	Stats about this Discord server
/userinfo [member]	Info about a member

Small tools

Command	What it does
/afk [reason]	Tell people you're away
/avatar [member]	Show someone's avatar full size
/choose <options>	Pick one of several options
/firstmessage [channel]	Jump to the start of a channel
/membercount	Member breakdown
/roleinfo <role>	Details about a role
/roll [dice]	Roll dice
/snipe	Show the last deleted message here
/stealemoji <emoji> [name]	Copy an emoji into this server
/timestamp [when] [style]	Build a Discord timestamp

Minigames

Command	What it does
/guessnumber [low] [high]	Start a guess-the-number game
/higherlower	Play higher or lower

Twitich

Command	What it does
<code>/twitch add <streamer> <channel> [role] [message]</code>	Announce when a Twitch streamer goes live
<code>/twitch list</code>	Streamers watched in this server
<code>/twitch remove <streamer></code>	Stop watching a streamer
<code>/twitch test <streamer></code>	Preview the announcement for a streamer

Partner

Command	What it does
<code>/partner browse</code>	See servers open to partnering
<code>/partner config</code>	Show this server's partner settings
<code>/partner list</code>	Show this server's partners
<code>/partner remove <server></code>	End a partnership
<code>/partner request <server></code>	Ask another server to partner
<code>/partner setup <channel> <invite> <description></code>	Configure this server's partner listing
<code>/partner toggle <enabled></code>	Enable or disable partnering here

Appearance

Command	What it does
<code>/serverprofile avatar [image] [url]</code>	Set the bot's avatar in this server only
<code>/serverprofile nickname [nickname]</code>	Set the bot's nickname in this server
<code>/serverprofile reset</code>	Restore the bot's default look here
<code>/serverprofile view</code>	Show this server's profile settings

General

Command	What it does
<code>/economy [pages]</code>	Total money held by the top players
<code>/help</code>	What DonutDuck can do
<code>/ping</code>	Check bot and API latency
<code>/status</code>	DonutSMP server status

Documentation

Command	What it does
/docs [section] [search]	Full documentation for every command

This guide was generated from the bot's own command tree, so the reference above matches the code exactly.